COMP 424 Final Project Game: *Ataxx*

Team Members: Jason Omer, Mert Cezar, Ali Meguellati
Due Date: Friday Dec 1st, 2025, 8:59PM EST
Report Due Date: Wednesday, Dec 3, 2025 8:59PM EST
Source for this file: `https://www.overleaf.com/project/68e2b2f07781caef1e11d2bc`

1. Executive Summary

The most effective algorithm found is Iterative-Deepening Minimax with Alpha-Beta pruning, supported by a simple move ordering. At a high level, the agent repeatedly deepens the search tree while enforcing a strict time limit of 2 seconds per move. At each node, it performs a standard minimax search with Alpha-Beta pruning and evaluates leaf positions using the material difference (the agent's discs minus the opponent's). In order to keep the branching factor manageable, the agent uses a one-step look ahead and expands only the top ten moves that flip the most opponent discs. This combination of cheap evaluation and move ordering allows Alpha-Beta to prune aggressively and reach useful depths within the time budget.

Performance-wise, this simpler agent outperforms all the other implemented agents. Search depth, time control, and a fast, stable evaluation give better results than any of the exotic heuristics tested. Earlier agents with more features struggled in practice: on a 7x7 board with a 2-second limit, the overhead of computing all of those features reduced effective search depth and sometimes caused unstable evaluations. By contrast, the simpler Alpha-Beta agent spends almost all of its time in the core search, with a very cheap material evaluator and a single but very useful heuristic. Play quality was assessed by pitting different versions of the agents against each other and against the baseline agents. Across these matches, the final Alpha-Beta version consistently delivered the strongest practical performance while also satisfying all constraints.

2. Detailed Explanation

2.1 Alpha-Beta Pruning

Of the many strategies tested, including Monte Carlo Tree Search, Alpha-Beta Pruning proved to be the most successful for the Ataxx game supporting the findings in *Using Monte Carlo Techniques in the Game Ataxx* paper [1]. This variant of Minimax works very well for finite zero-sum games. While it explores all branches, it minimizes search time by eliminating unnecessary searches.

Alpha-Beta Pruning has been implemented using two functions: *max_value* and *min_value*. When the *step* function of the agent is called, a list of valid moves is built. Every move in that list is then executed on a copy of the board, which is passed to *min_value*. *min_value* takes as arguments a board, values for α and β, and a depth limit. α and β are initially negative and positive infinity, respectively. If the board passed to *min_value* is a terminal state or the depth limit is attained, the utility value of the board is returned (see next section). Otherwise, all possible moves of the opponent player are generated and inserted in a list. Once again, in a loop, every possible move for the opponent is executed on a copy of the board, which is then passed to the *max_value* function with a decremented depth limit. β is then set to the minimum returned value from *max_value*. If at any point β becomes smaller than α, the loop is aborted and α is returned. This corresponds to

the pruning part, where the search is cut short when the current max player move is unlikely to be chosen. If the loop completes, then β, corresponding to the lowest possible utility of the branch, is returned.

On the other hand, the *max_value* function is very similar to *min_value*. Here, instead of updating β, α gets updated and returned after simulating all possible moves of the agent and finding the maximum utility. One can think of the *min_value* and *max_value* functions as two opposing players, each playing optimally in an alternating pattern until the given depth limit for the search tree is attained. The goal is to return the minimal achievable utility for the moves listed before the first call to *min_value*.

2.2 Utility

More sophisticated utility functions can offer additional benefits, but a simple board-score evaluation revealed sufficient for the Ataxx game. It provides an effective way to compare moves while keeping computational overhead low. Moreover, when terminal states appear in the search tree, the board score remains the most objective and reliable method for assessing move utility.

2.3 Time Management

While Ataxx has a finite search space, the requirement to select a move within a two-second time limit and to use only a single thread makes it impossible to explore all branches of the game tree, even with Alpha-Beta pruning. For that reason, few additions were made to the program.

A timer was added to the agent class and activates when the *step* function is called. Before exploring each node, the elapsed time is checked. When the time limit is close to being reached, the program aborts by raising an exception that is caught by the *step* function, which then returns the best move found so far. To increase the likelihood of exploring high-quality moves early, the available actions are sorted in decreasing order of their estimated utility at every level of the tree. The utility is approximated using the provided *count_disc_count_change* function, which computes the number of opponent discs that would be captured by a given move. This move ordering at every level also improves the efficiency of Alpha–Beta Pruning. By placing the highest-utility moves, whether for the max player or the min player, in the left branches of the search tree, pruning is triggered earlier.

Early experiments showed that increasing the depth limit of the search dramatically reduces the agent's success rate. Further analysis provided an explanation for this phenomenon: increasing the search depth makes it more likely for the timer to abort the program after it has explored only the leftmost branches of the tree, which in some cases do not include the optimal moves. Iterative deepening search solves this problem. By performing multiple iterations of Alpha–Beta search with increasing depth limits, the program ensures that more branches are expanded at shallower depths before the algorithm proceeds deeper. In other words, iterative deepening search maintains an adequate breadth in the search. This technique was implemented using a function that calls *min_value* with a gradually increasing depth limit. A dictionary containing the utility of each move at depth zero is created in iterative deepening search function and updated after every iteration of the Alpha–Beta search. This dictionary is then used to sort the moves of depth zero before the next iteration of the search. Only after every complete iteration of Alpha-Beta search, the best move for the agent gets updated, ensuring a fair comparison between all possible moves.

After testing the agent against human players and other agents, it became evident that exploring all nodes at every level was unnecessary and resulted in a very shallow search. A better balance was achieved by limiting the branching factor of each node to 10. This limit, combined with move reordering, allows the algorithm to reach depths of 6 or even 7 while still exploring sufficiently diverse branches.

3. Quantitative Analysis

3.1 Depth

The agent achieves a consistent look-ahead depth of 5 to 6 plies, occasionally reaching 7 plies in the late game. A ply counts for a single move by one player, so a depth of 6 plies corresponds to only 3 full turns. Depth varies because the efficiency of Alpha-Beta pruning depends heavily on the number of legal moves at each ply. The agent's move-ordering heuristic, which sorts moves by immediate disc gain, significantly improves pruning and allows deeper search within the fixed time budget.

3.2 Breadth

The program caps the effective branching factor at 10 moves per ply, selecting only the most promising moves using heuristic ordering. This dramatically reduces the search tree size, enabling Alpha-Beta to reach deeper levels. Raw branching varies wildly across the different boards, but the capped branching ensures both the max and min players explore roughly the same breadth.

3.3 Impact of Board Layouts

Board	Mean Depth	Std. Depth	Mean Raw Branching
empty_7x7	5.91	0.32	28–35
big_x	6.08	0.28	18–25
the_circle	5.84	0.30	30–40
point4	6.03	0.27	20–23
plus1	5.98	0.29	22–30
plus2	5.99	0.31	25–32
the_wall	5.81	0.35	35–42
watch_the_sides	6.02	0.26	21–25

Table 1: Search depth and branching characteristics of the agent across different board layouts. Effective branching is capped at 10 for all boards (not shown).

The board layout has a significant effect on the agent's search behaviour. Open boards generate many legal moves, which increases the raw branching and reduces achievable depth to around 5 plies. Boards with walls or bottlenecks constrain the number of moves enabling more Alpha-Beta pruning and allowing depth 6-7 searches. Because the agent's breadth is capped at 10 moves, different layouts mainly affect when pruning begins rather than the number of moves explicitly considered.

3.4 Heuristics, Pruning and Move Ordering

The final design employs iterative deepening, Alpha-Beta pruning, and a lightweight move-ordering heuristic based on immediate disc gain. Through extensive experimentation, it was found that this minimalist approach significantly outperforms earlier more complex agents that incorporate more advanced features. While these advanced features are theoretically powerful, the computational overhead they introduce substantially hinders the effective search depth and destabilizes pruning efficiency.

Before adopting this streamlined design, experimentation included a game-phase-dependent linear evaluation function that blended several heuristics: mobility, frontier exposure, center control, and material, each weighted differently depending on the game state (early vs late). Although this model was more strategic, each heuristic required multiple passes over the board, introducing overhead and limiting search depth. In practice, the resulting evaluation were sometimes unstable or biased in unintended ways, making the agent both slower and less reliable.

In contrast, the current implementation evaluates positions using only material difference and limits the branching factor to the most promising moves. This algorithm consistently reaches search depths of 5-6 plies, providing deeper tactical insight within the same 2 second limit.

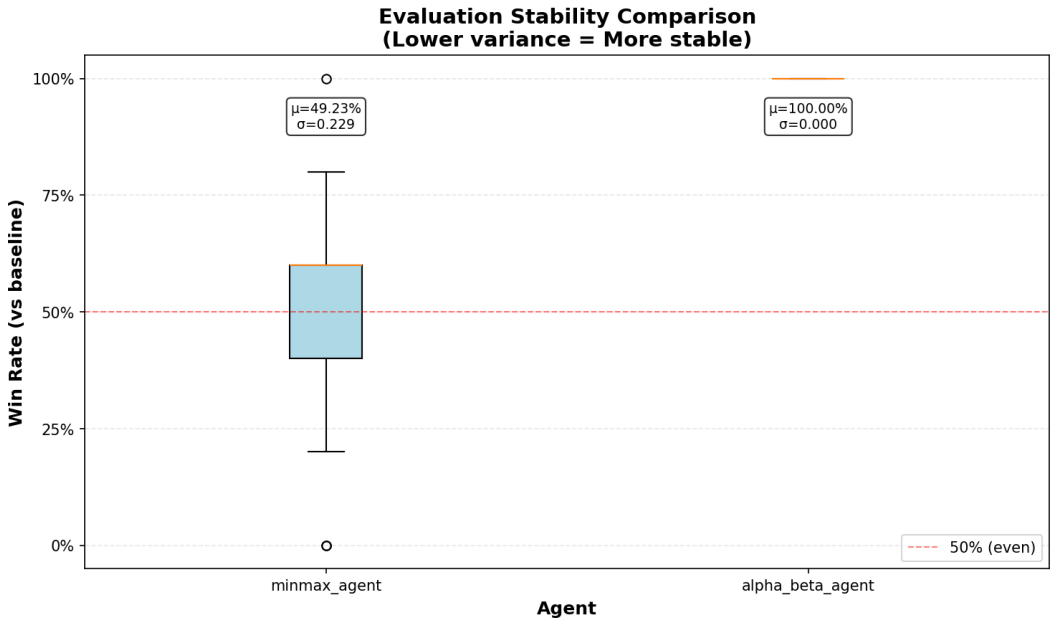

Figure 1: Evaluation stability comparison between earlier (left) and current (right) agent designs. Lower variance indicates more consistent decision-making.

Win-rate distributions were collected for both agents across all board layouts and against the same baseline opponent (`greedy_corners_agent`). The older `minmax_agent` (left) shows very high variance in its game outcomes, reflecting unstable evaluation and inconsistent pruning behavior. In contrast, the new `alpha_beta_agent` (right) exhibits no variance and a 100% win rate across all tests, confirming that the simplified evaluation function and improved pruning result in far more stable and reliable play.

3.5 Win Rate Predictions

Against the random agent, the expected win rate is 100% since even a very naive implementation of minimax can take advantage of the random agent's uncoordinated moves. Against an average human player, the expected win rate is 70-80%, and 100% if the human is also subjected to the same time constraints. As for other agents, performance will likely vary, but against a well-optimized Alpha-Beta agent with a similar implementation, the expected win rate is 50-70%.

4. Pros/cons of Chosen Approach

The final agent's deliberate minimalist design outperforms all of our more complex agents. The key advantage of the chosen approach is the computational budget that goes almost entirely into search depth, since both the evaluation and the move ordering heuristic are extremely fast. In practice, this deeper and more stable search proves to be more valuable than any sophisticated heuristic.

Another advantage of the final agent is its predictability and stability. A simpler agent is less likely to suffer from overfitting to specific board layouts, or unwanted interactions between multiple heuristics. The small branching factor also makes the agent's behavior more consistent across different board types.

The final agent's minimalist design also introduces some limitations. The material-only evaluation has no direct long-term awareness of concepts such as mobility (how many moves each player will have in future turns), enclosure patterns, or long-term setups. In other words, tactically valuable moves that sacrifice discs in the short term may be pruned if they don't immediately flip many discs.

5. Future Improvements

Potential improvements to the agent build on some computational techniques and higher search efficiency. The current Alpha Beta agent is limited to a maximum search depth of 10 moves in all game states. An alternative approach would be making this limit dependent on remaining time, board complexity, and availability of valid moves. This allows deeper searches in constrained positions and broader searches in open positions.

In addition, the Alpha Beta agent runs on a single thread due to competition constraints. In a more relaxed tournament, parallelizing the search using multiple cores can allow search depth to increase. This would improve pruning effectiveness. Another issue with the tournament environment is that the server's load varies as there are many processes being executed simultaneously. This can cause partial resource starvation. Assigning higher priority to the agent's process could mitigate this issue.

For late game positions, precomputed table bases (boards) guarantee optimal play. Such an approach eliminates uncertainty in the final moves at the expense of space complexity. Furthermore, the final chosen agent evaluates board states even if the same board states appeared in earlier search steps. Such repetitions are common in Ataxx-like games due to the small board size and limited number of valid moves in end game positions. Introducing caching can reduce repetitive exploration and free time for exploring undiscovered branches/depths.

The Alpha Beta agent selects moves by relying on disk gain heuristic. Which is effective. Different heuristic functions can improve performance based on other parameters (state of the board, opponent, etc.) Another approach may involve a machine learning model trained on previously played games, which can change this ordering by predicting moves that are promising based on previous game data.

Moreover, the agent breaks ties by selecting the first move with highest utility. For example, if node A was discovered with utility 10 and then later node B with utility 10. The agent selects A. A possible change can be to perform a brief random simulation to break ties (if time allows); this would yield a more balanced selection.

The suggestions discussed earlier point to a phase based strategy policy implementation for the agent, such as early and late game dynamics. In the early game phase, constraints are limited and the board is open, techniques such as dynamic move limit adjustment, improved move ordering, or machine learning heuristic in this phase can maintain broad exploration with promising move selection. In contrast, the late game phase benefits heavily from caching, reduced branching, and from table bases. Together, these enhancements would offer a stronger and more adaptive Ataxx agent.

6. Significant Input Resources

Claude AI and the university version of Copilot were only used to correct a few sentences in this report. Here are few example prompts and the output generated.

- "In a more relaxed tournaments" → should be "In more relaxed tournaments".

- "Which might lead to partial starvation" → better phrased as "This can cause partial resource starvation".

- "Prioritizing the agent's process would lead to better results" → clarify: "Assigning higher priority to the agent's process could mitigate this issue."

References

[1] Evert Cuppen. Using monte carlo techniques in the game ataxx. Bachelor's Paper, Department of Knowledge Engineering, Maastricht University, 2007. URL `https://project.dke.maastrichtuniversity.nl/games/files/bsc/Cuppen_BSc-paper.pdf`.